Phase_01

```
0000000000402020 <phase_01>:
402020: endbr64
402024: push   %rbp     ;保存调用函数的%rbp值,便于函数返回时恢复
402025: mov    %rsp,%rbp     ;构建栈帧,通过%rbp定位变量
402028: push   %r12     ;保存调用函数的%r12值,用它来保存challenge
40202a: mov    %rsi,%r12     ;将challenge赋值给%r12
40202d: mov    $0x1,%esi     ;将%esi的值设为1,用于检查输入是否合法
402032: push   %rbx     ;保存调用函数的%rbx值,用它来保存字符串地址
402033: mov    %rdi,%rbx     ;将字符串地址保存到%rbx
402036: sub    $0x10,%rsp     ;在栈上预留16字节空间

40203a: call   402bf0 <valid_u32_fields>     ;调用函数valid_u32_fields,检查输入格式
40203f: test   %al,%al     ;进行al&al运算，判断函数返回值是否为0
402041: jne    402050 <phase_01+0x30>     ;返回值非零表示合法,跳转到402050继续解析

402043: add    $0x10,%rsp     ;如果不合法,则回收预留的16字节栈空间
402047: xor    %eax,%eax     ;将%eax归零
402049: pop    %rbx     ;恢复调用函数原来的%rbx
40204a: pop    %r12     ;恢复调用函数原来的%r12
40204c: pop    %rbp     ;恢复调用函数原来的%rbp
40204d: ret    ;返回调用函数,并返回%eax=0,表示本关未通过

40204e: xchg   %ax,%ax
402050: xor    %eax,%eax     ;为调用sscanf做准备
402052: lea    -0x14(%rbp),%rdx     ;将局部变量的地址作为第三个参数,让sscanf把输入整数写在这里
402056: mov    $0x403182,%esi     ;将格式字符串地址作为sscanf的第二个参数
40205b: mov    %rbx,%rdi     ;将之前的字符串地址作为sscanf的第一个参数
40205e: call   4011e0 <__isoc99_sscanf@plt>     ;从输入字符串中解析整数,成功赋值的项目数由%eax返回
402063: cmp    $0x1,%eax     ;检查sscanf是否完成了一次赋值
402066: jne    402043 <phase_01+0x23>     ;如果没有赋值成功,跳到失败路径,函数返回假

402068: mov    %r12,%rdi     ;如果赋值成功,将challenge的值存储在%rdi中,准备计算正确口令
40206b: call   401fe0 <p1_compute>     ;根据challenge计算正确口令,结果保存在%eax中
402070: cmp    %eax,-0x14(%rbp)     ;比较栈上保存的用户输入与正确口令
402073: sete   %al     ;若二者相等则将%al设为1,不相等设为0
402076: add    $0x10,%rsp     ;回收栈空间
40207a: pop    %rbx
40207b: pop    %r12
40207d: pop    %rbp
40207e: ret    ;返回%al的结果
```

```
0000000000401fe0 <p1_compute>:
  401fe0: endbr64
  401fe4: push   %rbp     ;保存调用函数的%rbp值
  401fe5: mov    %rdi,%rax     ;将challenge复制在%rax,用于提取高32位
  401fe8: mov    %rdi,%rdx     ;将challenge复制在%rdx,用于提取右移16位后的数据
  401feb: mov    %edi,%r8d     ;将challenge的低32位保存在%r8d中,下面记为low
  401fee: shr    $0x20,%rax     ;将challenge逻辑右移32位,%rax中的值记为high
  401ff2: shr    $0x10,%rdx     ;将challenge逻辑右移16位,%rdx中的值记为mid
  401ff6: mov    %eax,%r9d     ;将high保存到%r9d中
  401ff9: mov    %eax,%esi     ;将high保存到%esi中,为调用p1_step0做准备
  401ffb: mov    %rsp,%rbp     ;设置当前的栈帧基准

  401ffe: call   401fb0 <p1_step0>     ;调用p1_step0,challenge和high的计算结果x返回到%eax
  402003: mov    %rdx,%rsi     ;将challenge>>16作为p1_step1的第二个参数,低32位为mid
  402006: mov    %r9d,%edi     ;将high作为第一个参数
  402009: mov    %eax,%ecx     ;将第一次的计算结果保存在%ecx中

  40200b: call   401fc0 <p1_step1>     ;调用p1_step1,high和mid的计算结果y返回到%eax
  402010: mov    %r8d,%esi     ;将low作为第二个参数
  402013: mov    %eax,%edi     ;将y作为第一个参数

  402015: call   401fd0 <p1_step2>     ;调用p1_step2,y和low的计算结果z返回到%eax
  40201a: pop    %rbp     ;恢复调用函数的%rbp
  40201b: xor    %ecx,%eax     ;计算z^x,作为最终正确口令
  40201d: ret    ;返回%eax中的32位口令
```

P1_step0:uint32_t x = (low + 2u * high) ^ 0xe2f0u;

P1_step1:uint32_t y = (mid ^ 0xd243u) + 2u * high;

P1_step2:uint32_t z = 3u * (y - low) + 0x4106u;

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ make
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challenge.c cli.c -o defuser
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 42 --phase 1
2353181703
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./bomb --seed 42 --phase 1
──────────────────────────────────────────────────────────────────────
BOMBLAB 2026                                  这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────────────────

单关调试：只记录本次通过的关卡，不播放完整结局。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
2353181703
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 1
2209638409
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./bomb --seed 43 --phase 1
```



Phase_02

```
402084: push   %rbp    ;保存调用函数的%rbp
402085: mov    %rsp,%rbp    ;建立栈帧基准
402088: push   %r13    ;保存调用函数的%r13,用它来保存challenge
40208a: mov    %rsi,%r13    ;将challenge保存到%r13中
40208d: mov    $0x6,%esi    ;将%esi设为6,检查是否包含六个整数
402092: push   %r12    ;保存调用函数的%r12,用它来保存输入字符串的地址
402094: mov    %rdi,%r12    ;将字符串地址保存到%r12
402097: push   %rbx    ;保存调用函数的%rbx
402098: sub    $0x28,%rsp    ;预留40字节局部空间

40209c: call   402bf0 <valid_u32_fields>    ;运行valid_u32_fields,检查格式是否符合32位无符号整数
4020a1: test   %al,%al    ;检查返回值是否为0
4020a3: jne    4020b8 <phase_02+0x38>    ;返回值非零表示格式合法,跳转到解析部分

4020a5: xor    %ebx,%ebx    ;将%ebx状态设为0,表示失败
4020a7: lea    -0x18(%rbp),%rsp    ;将%rsp恢复到old_rbx的位置,丢弃40位的局部空间
4020ab: mov    %ebx,%eax    ;将成功或失败的结果放入返回值寄存器
4020ad: pop    %rbx    ;恢复调用函数的%rbx
4020ae: pop    %r12    ;恢复调用函数的%r12
4020b0: pop    %r13    ;恢复调用函数的%r13
4020b2: pop    %rbp    ;恢复调用函数的%rbp
4020b3: ret    ;返回结果
4020b4: nopl   0x0(%rax)

;将六个输入整数记为a[0]~a[5]
4020b8: mov    %eax,%ebx    ;保存格式检查返回的成功状态,若后续检查均通过则返回该值
4020ba: lea    -0x2c(%rbp),%rax    ;将a[5]的地址保存在%rax中
4020be: lea    -0x3c(%rbp),%rcx    ;将a[1]的地址保存在%rcx中
4020c2: mov    $0x403173,%esi    ;将格式字符串的地址作为sscanf的第二个参数
4020c7: push   %rax    ;将&a[5]压栈,作为sscanf的第八个参数
4020c8: lea    -0x30(%rbp),%rax    ;将&a[4]保存在%rax中
4020cc: lea    -0x40(%rbp),%rdx    ;将&a[0]保存在%rdx中
4020d0: mov    %r12,%rdi    ;将输入字符串地址作为第一个参数
4020d3: push   %rax    ;将&a[4]压栈,作为第七个参数
4020d4: lea    -0x34(%rbp),%r9    ;将&a[3]保存在%r9中
4020d8: lea    -0x38(%rbp),%r8    ;将&a[2]保存在%r8中
4020dc: xor    %eax,%eax    ;将%eax设为0
4020de: call   4011e0 <__isoc99_sscanf@plt>    ;调用_isoc99_sscanf@plt,从输入字符串中解析出六个
4020e3: pop    %rdx    ;回收栈参数%rdx
```

```
4020e4: pop    %rcx    ;回收栈参数%rcx
4020e5: cmp    $0x6,%eax    ;检查sscanf是否完成六次赋值
4020e8: jne    4020a5 <phase_02+0x25>    ;如果没有完成,跳转到失败路径

4020ea: mov    %r13,%rax    ;将challenge复制到%rax
4020ed: mov    -0x40(%rbp),%esi    ;读取用户输入的a[0],并赋值给%esi
4020f0: shr    $0x10,%rax    ;将challenge逻辑右移16位
4020f4: and    $0xf,%eax    ;保留最低四位
4020f7: xor    $0x10,%eax    ;将结果与16异或,得到a[0]的正确值
4020fa: cmp    %eax,%esi    ;比较a[0]和正确值
4020fc: jne    4020a5 <phase_02+0x25>    ;如果不相等,跳转到失败路径

4020fe: lea    -0x40(%rbp),%rdx    ;将&a[0]赋值给%rdx,作为遍历数组的指针
402102: mov    $0x18,%r8d    ;将24保存在%r8d中
402108: mov    $0x6,%edi    ;将递推的加数初始化为6
40210d: sub    %edx,%r8d    ;计算24-数组首地址的低32位
402110: lea    (%r8,%rdx,1),%ecx    ;计算位移量:初始为24,每次前进4字节,移位量增加4
402114: mov    %r13,%rax    ;每轮重新复制原始challenge
402117: add    %esi,%esi    ;将当前元素乘以2
402119: shr    %cl,%rax    ;将challenge右移cl指定的位数
40211c: and    $0x3f,%eax    ;保留移位结果的最低6位
40211f: xor    %esi,%eax    ;将提取出的6位数据与2*a[i-1]异或
402121: mov    0x4(%rdx),%esi    ;读取下一个输入元素a[i]
402124: add    %edi,%eax    ;加上本轮加数,依次为6 12 18 24 30
402126: cmp    %eax,%esi    ;比较a[i]与正确值
402128: jne    4020a5 <phase_02+0x25>    ;若不相等,跳转到失败路径

40212e: add    $0x6,%edi    ;加数增加6
402131: add    $0x4,%rdx    ;指针移动到下一个数组元素
402135: cmp    $0x24,%edi    ;判断加数是否达到36,即五轮循环是否结束
402138: jne    402110 <phase_02+0x90>    ;若尚未结束,则继续下一轮检查
40213a: jmp    4020a7 <phase_02+0x27>    ;全部检查通过后,跳转到结束路径
```

```
// 第一个数：提取 challenge 的第 16～19 位，再与 16 异或。
values[0] = (uint32_t)((challenge >> 16) & 0xfu) ^ 0x10u;
// 后五个数：根据前一个数、challenge 中的位段和本轮加数递推。
for (uint32_t i = 1; i < 6u; ++i) {
    uint32_t shift = 20u + 4u * i;
    uint32_t part = (uint32_t)((challenge >> shift) & 0x3fu);
    values[i] = (part ^ (2u * values[i - 1])) + 6u * i;
}
count = 6;
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 42 --phase 2 | ./bomb --seed 42 --phase 2
BOMBLAB 2026                                        这里还没有明天
```




```
[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 2 | ./bomb --seed 43 --phase 2
BOMBLAB 2026                                        这里还没有明天
```




```
[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

Phase_03

```
402220: endbr64
402224: push   %rbp     ;保存调用函数的%rbp
402225: mov    %rsp,%rbp     ;建立栈帧基准
402228: push   %r12     ;保存调用函数的%r12,用它保存challenge
40222a: mov    %rsi,%r12     ;将challenge保存到%r12
40222d: mov    $0x2,%esi     ;将第二个参数设为2,检查输入是否包含两个整数
402232: push   %rbx     ;保存调用函数的%rbx
402233: mov    %rdi,%rbx     ;将输入字符串地址保存到%rbx
402236: sub    $0x10,%rsp     ;预留16字节的局部空间
40223a: call   402bf0 <valid_u32_fields>     ;检查输入是否符合两个无符号的32位整数
40223f: test   %al,%al     ;检查结果是否为0
402241: jne    402250 <phase_03+0x30>     ;若非零则合法,跳转到解析部分

402243: xor    %eax,%eax     ;将返回值设为0,表示本关未通过
402245: add    $0x10,%rsp     ;回收16字节局部空间
402249: pop    %rbx     ;恢复调用函数的%rbx
40224a: pop    %r12     ;恢复调用函数的%r12
40224c: pop    %rbp     ;恢复调用函数的%rbp
40224d: ret    ;返回测试结果
40224e: xchg   %ax,%ax

;将两个输入整数记为a[0],a[1]
402250: xor    %eax,%eax     ;将%eax设为0
402252: lea    -0x14(%rbp),%rcx     ;将a[1]的起始地址赋值给%rcx
402256: lea    -0x18(%rbp),%rdx     ;将a[0]的起始地址赋值给%rdx
40225a: mov    $0x40317f,%esi     ;将格式字符串地址赋值给%esi
40225f: mov    %rbx,%rdi     ;将输入字符串地址赋值给%rdi
402262: call   4011e0 <__isoc99_sscanf@plt>     ;解析两个整数,成功赋值的项目数返回%eax
402267: cmp    $0x2,%eax     ;检测是否完成两次赋值
40226a: jne    402243 <phase_03+0x23>     ;若没有解析成功,则跳转到失败路径

40226c: mov    %r12d,%edi     ;将challenge的低32位复制到%edi
40226f: xor    %eax,%eax     ;将%eax设为0
402271: and    $0x7,%edi     ;保留最低三位,得到正确值
402274: cmp    %edi,-0x18(%rbp)     ;比较用户输入的a[0]和正确值
402277: jne    402245 <phase_03+0x25>     ;如果不相等,则跳转到失败路径

402279: mov    %r12,%rax     ;将challenge复制到%rax
40227c: mov    %r12,%rsi     ;将challenge复制到%rsi
40227f: shr    $0x3,%rax     ;将challenge右移3位
402283: shr    $0x20,%rsi     ;将challenge右移32位
402287: xor    %eax,%esi     ;两个移位结果的低32位异或
402289: call   4021c0 <p3_dispatch>     ;根据edi中的index选择分支,以esi中的x参与计算
40228e: cmp    %eax,-0x14(%rbp)     ;比较返回结果与a[1]
402291: sete   %al     ;如果相等,则将%al设为1,否则设为0
402294: jmp    402245 <phase_03+0x25>     ;跳转返回部分,保留比较结果
```

```
4021c0: endbr64
4021c4: mov    %edi,%eax    ;将第一个参数index保存到%eax,作为跳转下标
4021c6: mov    %esi,%edi    ;将第二个参数x保存到%edi,作为各个分支函数的第一个参数
4021c8: cmp    $0x7,%eax    ;将index和7比较
4021cb: ja     402215 <p3_dispatch+0x55>    ;若index大于7,跳转到默认返回路径

4021cd: notrack jmp *0x4057e0(,%rax,8)    ;从地址0x4057e0+index*8读取目标地址,并跳转过去
4021d5: nopl   (%rax)

4021d8: jmp    402190 <p3_case_5>
4021da: nopw   0x0(%rax,%rax,1)

4021e0: jmp    402140 <p3_case_0>
4021e5: nopl   (%rax)

4021e8: jmp    402150 <p3_case_1>
4021ed: nopl   (%rax)

4021f0: jmp    402170 <p3_case_3>
4021f5: nopl   (%rax)

4021f8: jmp    4021a0 <p3_case_6>
4021fa: nopw   0x0(%rax,%rax,1)

402200: jmp    4021b0 <p3_case_7>
402202: nopw   0x0(%rax,%rax,1)

402208: jmp    402180 <p3_case_4>
40220d: nopl   (%rax)

402210: jmp    402160 <p3_case_2>

402215: xor    %eax,%eax    ;index越界时返回0
402217: ret
```

case 0:

values[1] = (x ^ 0x3f27u) + (x >> 1);

break;

case 1:

values[1] = 5u * x + 0x48143u;

break;

case 2:

```
values[1] = 3u * x + 0x7e69u;
break;

case 3:
values[1] = (x ^ 0xcd68u) + (x >> 1);
break;

case 4:
values[1] = (2u * x) ^ 0x1f73u;
break;

case 5:
values[1] = 5u * x + 0xc173u;
break;

case 6:
values[1] = 5u * x + 0x1cebcu;
break;

case 7:
values[1] = (2u * x) ^ 0x508au;
break;
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 42 --phase 3 | ./bomb --seed 42 --phase 3
BOMBLAB 2026                                        这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x581ce1ff0e4ae394
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 3 | ./bomb --seed 43 --phase 3
BOMBLAB 2026                                        这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]
```

Phase_04

```
402310: endbr64
402314: push   %rbp
402315: mov    %rsp,%rbp
402318: push   %r12
40231a: mov    %rsi,%r12     ;将challenge保存到%r12
40231d: mov    $0x1,%esi     ;将第二个参数设为1,检查一个整数的输入格式
402322: push   %rbx
402323: mov    %rdi,%rbx     ;将输入字符串地址保存到%rbx
402326: sub    $0x10,%rsp     ;预留16字节的局部空间
40232a: call   402bf0 <valid_u32_fields>     ;检查输入是否符合一个无符号32位整数的格式
40232f: test   %al,%al     ;检查格式检查结果是否为0
402331: jne    402340 <phase_04+0x30>     ;格式合法则跳转到解析部分

402333: add    $0x10,%rsp     ;回收16位局部空间
402337: xor    %eax,%eax     ;将%eax设为0,表示失败
402339: pop    %rbx
40233a: pop    %r12
40233c: pop    %rbp
40233d: ret    ;返回测试结果
40233e: xchg   %ax,%ax

402340: xor    %eax,%eax
402342: lea    -0x14(%rbp),%rdx     ;接收输入整数的变量地址
402346: mov    $0x403182,%esi     ;将格式字符串%u的地址保存到%esi
40234b: mov    %rbx,%rdi     ;将输入字符串地址保存到%rdi
40234e: call   4011e0 <__isoc99_sscanf@plt>     ;将输入解析为一个整数
402353: cmp    $0x1,%eax     ;检查是否成功完成一次赋值
402356: jne    402333 <phase_04+0x23>     ;若未完成,则跳转到失败路径
```

```
402358: mov    %r12,%rcx     ;复制challenge,用于计算第四个参数
40235b: mov    %r12,%rax     ;复制challenge,用于提取高32位
40235e: mov    %r12d,%edi     ;将challenge的低32位保存到%edi
402361: xor    %esi,%esi     ;将第二个参数设为0
402363: shr    $0x20,%rax     ;%rax=challenge>>32
402367: shr    $0x7,%rcx     ;%rcx=challenge>>7
40236b: and    $0x7f,%edi     ;取challenge的最低7位
40236e: mov    $0x7f,%edx     ;第三个参数设为127
402373: xor    %eax,%ecx     ;两个移位结果的低32位异或
402375: call   4022a0 <p4_walk>     ;调用递归函数,结果返回到%eax

40237a: cmp    %eax,-0x14(%rbp)     ;比较输入整数和正确值
40237d: sete   %al     ;若相等则设为1,否则设为0
402380: add    $0x10,%rsp
402384: pop    %rbx
402385: pop    %r12
402387: pop    %rbp
402388: ret    ;返回测试通过的结果
```

```
;%edi=target
;%esi=low
;%edx=high
;%ecx=key

4022a0: endbr64
4022a4: mov    %edi,%eax     ;将target保存到%eax
4022a6: mov    %edx,%edi     ;%edi=high
4022a8: sub    %esi,%edi     ;%edi=high-low
4022aa: shr    $1,%edi     ;%edi=(high-low)/2
4022ac: add    %esi,%edi     ;%edi=low+(high-low)/2,即区间中点mid
4022ae: cmp    %eax,%edi     ;判断mid和target的大小
4022b0: je     402300 <p4_walk+0x60>     ;如果相等,则跳转到递归终止分支

4022b2: push   %rbp
4022b3: mov    %rsp,%rbp
4022b6: push   %rbx
4022b7: lea    (%rdi,%rcx,1),%ebx     ;%ebx=mid+key
4022ba: sub    $0x8,%rsp     ;预留8字节的局部空间
4022be: cmp    %edi,%eax     ;比较target和mid
4022c0: jb     4022e0 <p4_walk+0x40>     ;若target<mid,则转入左半区间

;右半区间
4022c2: lea    0x1(%rdi),%esi     ;low=mid+1
4022c5: mov    %eax,%edi     ;将target保存到%edi
4022c7: call   4022a0 <p4_walk>     ;递归调用

4022cc: lea    (%rax,%rax,4),%eax     ;子调用返回后,将返回值乘以5,保留低32位
4022cf: xor    %ebx,%eax     ;与mid+key异或
4022d1: mov    -0x8(%rbp),%rbx     ;恢复上一层的%rbx
4022d5: leave    ;清理本层栈帧
4022d6: xor    $0xd,%eax     ;将%eax与13异或,得到本层返回值
4022d9: ret    ;返回上一层
4022da: nopw   0x0(%rax,%rax,1)
```

```
;左半区间
4022e0: lea    -0x1(%rdi),%edx     ;high=mid-1
4022e3: mov    %eax,%edi     ;将target保存到%edi
4022e5: call   4022a0 <p4_walk>     ;递归调用

4022ea: lea    (%rax,%rax,4),%eax     ;子调用返回后,将返回值乘以5,并取低32位
4022ed: xor    %ebx,%eax     ;与mid+key异或
4022ef: mov    -0x8(%rbp),%rbx     ;恢复上一层的%rbx
4022f3: leave    ;清理本层栈帧
4022f4: xor    $0x1,%eax     ;将%eax与1异或,得到本层返回值
4022f7: ret    ;返回上一层
4022f8: nopl   0x0(%rax,%rax,1)

402300: lea    0x62(%rdi),%eax     ;%eax=mid+98
402303: xor    %ecx,%eax     ;将%eax和key异或
402305: ret    ;返回%eax值
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 4 | ./bomb --seed 43 --phase 4
BOMBLAB 2026                                    这里还没有明天

                    *
                     \
                  .-"""-.
                .'       '.
               /    LOOP    \
              |               |
               \             /
                '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ make
./defuser --seed 42 --phase 4 | ./bomb --seed 42 --phase 4
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challenge.c cli.c -o defuser
BOMBLAB 2026                                    这里还没有明天

                    *
                     \
                  .-"""-.
                .'       '.
               /    LOOP    \
              |               |
               \             /
                '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x09bc585a244823f2
Input (unsigned decimal integers):
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

Phase_05

```
402408: push   %r12
40240a: mov    %rsi,%r12     ;将challenge保存到%r12
40240d: mov    $0x2,%esi     ;将%esi设为2,检查两个整数的输入格式
402412: push   %rbx
402413: mov    %rdi,%rbx     ;将输入字符串的地址保存到%rbx
402416: sub    $0x10,%rsp     ;预留16位的局部空间
40241a: call   402bf0 <valid_u32_fields>     ;检查格式是否合法
40241f: test   %al,%al     ;判断返回值是否为0
402421: jne    402430 <phase_05+0x30>     ;若非零,则跳转解析部分

402423: xor    %eax,%eax     ;将返回值设为0,表示失败
402425: add    $0x10,%rsp
402429: pop    %rbx
40242a: pop    %r12
40242c: pop    %rbp
40242d: ret    ;返回测试结果
40242e: xchg   %ax,%ax

402430: xor    %eax,%eax     ;将%eax设为0
402432: lea    -0x18(%rbp),%rcx     ;接收输入第二个整数的地址
402436: lea    -0x1c(%rbp),%rdx     ;接收输入第一个整数的地址
40243a: mov    $0x40317f,%esi     ;将格式字符串%u %u的地址保存到%esi
40243f: mov    %rbx,%rdi     ;将输入字符串的地址保存到%rdi
402442: call   4011e0 <__isoc99_sscanf@plt>     ;从字符串中解析出两个整数
402447: cmp    $0x2,%eax     ;判断是否成功解析出两个整数
40244a: jne    402423 <phase_05+0x23>     ;若未解析成功,则跳转失败路径

40244c: lea    -0x14(%rbp),%rsi     ;接收last_id的地址
402450: mov    %r12,%rdi     ;将challenge复制到%rdi
402453: call   402390 <p5_traverse>     ;计算遍历结果
402458: mov    %eax,%edx     ;将返回的结果暂存到%edx
40245a: xor    %eax,%eax     ;将%eax设为0
40245c: cmp    %edx,-0x1c(%rbp)     ;比较用户输入的第一个整数和遍历计算结果
40245f: jne    402425 <phase_05+0x25>     ;若不相等,则跳转失败路径
```

```
402461: mov    -0x14(%rbp),%eax     ;读取遍历函数写回的last_id
402464: cmp    %eax,-0x18(%rbp)     ;比较第二个输入整数和与最后访问节点的编号
402467: sete   %al     ;若相等则设为1,否则设为0
40246a: jmp    402425 <phase_05+0x25>     ;跳转返回路径,返回测试结果
```

```
402390: endbr64
402394: mov    %rdi,%rax    ;复制challenge,准备计算起点下标
402397: mov    %rdi,%rcx    ;复制challenge,准备计算移动次数
40239a: mov    %rsi,%r8    ;将last_id的地址保存到%r8
40239d: and    $0x7,%eax    ;取challenge的最低3位
4023a0: shr    $0x3,%rcx    ;challenge右移3位,取其中最低2位作为移动次数
4023a4: mov    0x405b20(,%rax,8),%rdx    ;从头指针表读取一个8字节节点地址,作为遍历起点

4023ac: mov    %rdi,%rax    ;将challenge复制到%rax
4023af: shr    $0x20,%rdi    ;%rdi=challenge>>32
4023b3: shr    $0x6,%rax    ;%rax=challenge>>6
4023b7: xor    %edi,%eax    ;将两次移位结果的低32位异或
4023b9: mov    0x8(%rdx),%esi    ;读取起点节点的id
4023bc: mov    %ecx,%edi    ;将challenge>>3的低32位复制到%edi
4023be: xor    $0xb8a6,%eax    ;再与常量0xb8a6异或,得到初始计算状态
4023c3: lea    (%rax,%rax,2),%eax    ;将状态乘以3,保留低32位
4023c6: add    0xc(%rdx),%eax    ;加上当前节点的value
4023c9: mov    %esi,(%r8)    ;将当前节点的id写入last_id
4023cc: xor    %esi,%eax    ;将状态与当前与当前节点id异或,完成起点节点的处理
4023ce: and    $0x3,%edi    ;取最低2位,得到沿next移动的次数
4023d1: je     4023f0 <p5_traverse+0x60>    ;若移动次数为0,则直接返回

4023d3: mov    $0x1,%ecx    ;初始化循环计数,准备进行第一次移动
4023d8: mov    (%rdx),%rdx    ;读取当前节点的next指针,让 %rdx指向下一个节点
4023db: lea    (%rax,%rax,2),%eax    ;将当前状态乘以3,保留低32位
4023de: add    $0x1,%ecx    ;计数增加1,用于判断是否需要下一次移动
4023e1: mov    0x8(%rdx),%esi    ;读取新节点id
4023e4: add    0xc(%rdx),%eax    ;将新节点的value加入状态
4023e7: xor    %esi,%eax    ;与新节点id异或
4023e9: mov    %esi,(%r8)    ;用新节点id更新last_id
4023ec: cmp    %ecx,%edi    ;比较规定移动次数与更新后的计数
4023ee: jae    4023d8 <p5_traverse+0x48>    ;若规定次数大于等于计数,则继续移动
4023f0: ret    ;返回%eax中的状态
```

```
for (uint32_t i = 0; i <= steps; ++i) {
    state = (3u * state + nodes[current].value) ^ nodes[current].id;
    values[1] = nodes[current].id;
    if (i < steps) {
        current = nodes[current].next;
    }
}
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ make
./defuser --seed 42 --phase 5 | ./bomb --seed 42 --phase 5
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challenge.c cli.c -o defuser
```

```
BOMBLAB 2026                                    这里还没有明天
```




```
[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xde4431fa3c80db06
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 5 | ./bomb --seed 43 --phase 5
```

```
BOMBLAB 2026                                    这里还没有明天
```




```
[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]
```

Phase_06

```
402550: endbr64
402554: push   %rbp
402555: mov    %rsp,%rbp
402558: push   %r12
40255a: mov    %rsi,%r12     ;将challenge保存到%r12
40255d: mov    $0x1,%esi     ;将%esi设为1,检查一个输入整数的输入格式
402562: push   %rbx
402563: mov    %rdi,%rbx     ;将输入字符串保存到%rbx
402566: sub    $0x10,%rsp     ;预留16字节的局部空间
40256a: call   402bf0 <valid_u32_fields>     ;判断整数输入是否合法
40256f: test   %al,%al
402571: jne    402580 <phase_06+0x30>     ;若格式合法,则跳转解析部分

402573: add    $0x10,%rsp
402577: xor    %eax,%eax     ;将返回值设为0,表示失败
402579: pop    %rbx
40257a: pop    %r12
40257c: pop    %rbp
40257d: ret    ;返回测试结果
40257e: xchg   %ax,%ax

402580: xor    %eax,%eax
402582: lea    -0x14(%rbp),%rdx     ;接收输入整数的地址
402586: mov    $0x403182,%esi     ;接收格式字符串%u的地址
40258b: mov    %rbx,%rdi     ;接收输入字符串地址
40258e: call   4011e0 <__isoc99_sscanf@plt>     ;解析出一个输入整数
402593: cmp    $0x1,%eax     ;检查是否成功解析
402596: jne    402573 <phase_06+0x23>     ;若未解析成功,则跳转失败路径

402598: mov    %r12,%rdi     ;将challenge作为第一个参数
40259b: call   402500 <p6_dispatch>     ;计算正确口令,结果返回%eax
4025a0: cmp    %eax,-0x14(%rbp)     ;比较输入整数与正确口令
4025a3: sete   %al     ;若相等则设为1,否则设为0
4025a6: add    $0x10,%rsp
4025aa: pop    %rbx
4025ab: pop    %r12
4025ad: pop    %rbp
4025ae: ret    ;返回测试结果
```

```
402500: endbr64
402504: push   %rbp
402505: mov    %rsp,%rbp
402508: push   %rbx
402509: mov    %rdi,%rbx     ;将challenge保存到%rbx
40250c: shr    $0x6,%rdi     ;%rdi=challenge>>6
402510: mov    %rbx,%rax     ;复制一份原始challenge
402513: shr    $0x20,%rax     ;%rax=challenge>>32
402517: xor    %eax,%edi     ;两次移位结果的低32位异或,得到初始输入x
402519: mov    %rbx,%rax     ;再取出原始challenge,用于计算第一次调用的下标
40251c: sub    $0x8,%rsp     ;预留8字节空间
402520: and    $0x38,%ebx     ;保留challenge的3~5位
402523: and    $0x7,%eax     ;保留challenge的0~2位
```

```
402526: call   *0x405dc0(,%rax,8)     ;从表中读取第一个first函数地址,并调用:x在%edi中
40252d: xor    $0x36e5,%eax     ;将第一次调用的返回值与0x36e5异或
402532: mov    %eax,%edi     ;将异或结果作为第二次调用的第一个参数
402534: call   *0x405dc0(%rbx)     ;读取第二个函数地址并调用
40253a: mov    -0x8(%rbp),%rbx     ;恢复调用函数原来的%rbx
40253e: leave
40253f: add    $0xb354,%eax     ;将第二次调用结果+0xb354,得到最终口令
402544: ret    ;返回最终32位结果
```

P6_op_1:result = ((x >> 5) ^ x) + 0x17fau;

P6_op_2:result = ((x << 7) | (x >> 25)) ^ 0xfb0bu;

P6_op_3:result = (x >> 7) | (x << 25);

P6_op_4:result = (x ^ 0x6425u) + (x << 5);

P6_op_5:result = (x + 0x8b6eu) ^ (x >> 3);

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ make
./defuser --seed 42 --phase 6 | ./bomb --seed 42 --phase 6
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challenge.c cli.c -o defuser
BOMBLAB 2026                                  这里还没有明天

                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x37e9671c45376d5d
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ ./defuser --seed 43 --phase 6 | ./bomb --seed 43 --phase 6
BOMBLAB 2026                                        这里还没有明天

                                *
                                 \
                              .-"""-.
                            .'       '.
                           /    LOOP    \
                          |               |
                           \             /
                            '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```

Honor

```
00000000004026a0 <secret_gate>:
  4026a0: endbr64
  4026a4: push   %rbp
  4026a5: xor    %eax,%eax     ;将%rax清零,准备初始化局部内存
  4026a7: mov    $0x14,%ecx     ;设置重复次数为20
  4026ac: mov    %rsp,%rbp
  4026af: push   %r14
  4026b1: mov    %rdx,%r14     ;将context保存到%r14
  4026b4: push   %r13
  4026b6: mov    %rsi,%r13     ;将challenge保存到%r13
  4026b9: push   %r12
  4026bb: lea    -0xd0(%rbp),%r12     ;%r12指向局部复制区域的起始位置
  4026c2: push   %rbx
  4026c3: mov    %rdi,%rbx     ;将输入文本地址保存到%rbx
  4026c6: mov    %r12,%rdi     ;将复制区域起始地址保存到%rdi,准备清零
  4026c9: sub    $0xb0,%rsp     ;预留176字节局部空间
  4026d0: rep stos %rax,(%rdi)     ;重复写入20个8字节的0,共清零160字节
  4026d3: movl   $0x0,(%rdi)     ;再清零后面的4字节,合计164字节

  4026d9: mov    %rbx,%rdi     ;将输入文本地址作为strlen的第一个参数
  4026dc: call   401190 <strlen@plt>     ;计算字符串长度,不包括结尾的空字符
  4026e1: sub    $0x24,%rax     ;长度减去36
  4026e5: cmp    $0x7f,%rax     ;将减后结果与127比较
  4026e9: jbe    402700 <secret_gate+0x60>     ;小于或等于127时,继续检查

  4026eb: add    $0xb0,%rsp     ;回收局部空间
  4026f2: xor    %eax,%eax     ;将%eax设为0,表示验证失败
  4026f4: pop    %rbx
  4026f5: pop    %r12
  4026f7: pop    %r13
  4026f9: pop    %r14
  4026fb: pop    %rbp
  4026fc: ret
  4026fd: nopl   (%rax)
```

```
402700: mov    %rbx,%rsi     ;将输入文本地址作为第二个参数
402703: mov    %r12,%rdi     ;将局部复制区域地址作为第一个参数
402706: call   402680 <secret_copy>     ;将输入文本连同结尾空字符,复制到局部区域
40270b: cmpl   $0x4e45504f,-0xb0(%rbp)     ;检查%rbp-0xb0处的4字节是否等于指定常量
402715: jne    4026eb <secret_gate+0x4b>     ;若不相等,则跳转失败路径

402717: mov    %r14,%rdx     ;将context作为第三个参数
40271a: mov    %r13,%rsi     ;将challenge作为第二个参数
40271d: lea    -0xac(%rbp),%rdi     ;将从复制区域偏移36开始的文本作为第一个参数
402724: call   4025c0 <secret_phase>     ;验证后面的三个整数
402729: add    $0xb0,%rsp     ;回收局部空间
402730: pop    %rbx
402731: pop    %r12
402733: pop    %r13
402735: pop    %r14
402737: pop    %rbp
402738: ret    ;返回验证结果

0000000000402680 <secret_copy>:
  402680: endbr64
  402684: xor    %eax,%eax     ;字节下标i初始化为0
  402686: cs nopw 0x0(%rax,%rax,1)

  402690: movzbl (%rsi,%rax,1),%edx     ;读取源字符串的第i个字节,并零扩展到%edx
  402694: mov    %dl,(%rdi,%rax,1)     ;将这个字节写入目标区域的第i个位置
  402697: add    $0x1,%rax     ;字节下标增加1
  40269b: test   %dl,%dl     ;检查是否为空字符
  40269d: jne    402690 <secret_copy+0x10>     ;如果不是空字符,则继续复制
  40269f: ret    ;空字符也复制,返回函数
```

```
00000000004025c0 <secret_phase>:
  4025c0: endbr64
  4025c4: push   %rbp
  4025c5: mov    %rsp,%rbp
  4025c8: push   %r13
  4025ca: mov    %rdx,%r13     ;将context保存到%r13
  4025cd: push   %r12
  4025cf: mov    %rsi,%r12     ;将challenge保存到%r12
  4025d2: mov    $0x3,%esi     ;将第二个参数设为3,检查三个整数的格式
  4025d7: push   %rbx
  4025d8: mov    %rdi,%rbx     ;保存数字文本地址
  4025db: sub    $0x18,%rsp     ;预留24字节的局部空间
  4025df: call   402bf0 <valid_u32_fields>     ;检查数字文本是否包含3个合法整数
  4025e4: test   %al,%al
  4025e6: jne    402600 <secret_phase+0x40>     ;如果格式合法,则进入解析部分

  4025e8: xor    %edx,%edx     ;将返回状态设为0,表示失败
  4025ea: add    $0x18,%rsp
  4025ee: mov    %edx,%eax
  4025f0: pop    %rbx
  4025f1: pop    %r12
  4025f3: pop    %r13
  4025f5: pop    %rbp
  4025f6: ret
  4025f7: nopw   0x0(%rax,%rax,1)
```

```
402600: xor    %eax,%eax
402602: lea    -0x24(%rbp),%r8     ;将第三个输入整数的接收地址作为第五个参数
402606: lea    -0x28(%rbp),%rcx     ;将第二个输入整数的接收地址作为第四个参数
40260a: mov    $0x40317c,%esi     ;将格式字符串%u %u %u的地址作为第二个参数
40260f: lea    -0x2c(%rbp),%rdx     ;将第一个输入整数的接收地址作为第三个参数
402613: mov    %rbx,%rdi     ;将数字文本地址作为第一个参数
402616: call   4011e0 <__isoc99_sscanf@plt>     ;解析三个整数
40261b: cmp    $0x3,%eax     ;检查是否解析成功
40261e: jne    4025e8 <secret_phase+0x28>     ;若不成功,则跳转失败路径

402620: mov    %r13d,%ecx     ;复制context的低32位
402623: xor    %edx,%edx     ;预先设置失败状态为0
402625: and    $0x3,%ecx     ;取context的低2位
402628: cmp    %ecx,-0x2c(%rbp)     ;比较第一个输入整数与正确口令
40262b: jne    4025ea <secret_phase+0x2a>     ;若不相等,则跳转失败路径

40262d: mov    %r12,%rsi     ;复制challenge
402630: imul   $0x3348,%ecx,%r9d     ;%r9d=v0*0x3348,保留低32位
402637: mov    %r13,%r8     ;复制context,准备高低32位异或
40263a: shr    $0x20,%rsi     ;%rsi=challenge>>32
40263e: shr    $0x20,%r8     ;%r8=context>>32
402642: xor    %r12d,%esi     ;%esi=challenge的高32位异或低32位,记为A
402645: xor    %r13d,%r8d     ;%r8d=context的高32位异或低32位,记为B
402648: shr    $0x10,%r13     ;%r13=context>>16
40264c: mov    %esi,%edi     ;将A作为准备调用的参数
40264e: xor    %r13d,%r9d     ;%r9d=(v0*0x3348)^(context>>16的低32位)
402651: xor    %r8d,%edi     ;%edi=A^B
402654: call   4025b0 <secret_mix>     ;对A^B进行循环右移和异或
402659: xor    %r9d,%eax     ;将返回值与%r9d异或,得到v1的正确值
40265c: cmp    %eax,-0x28(%rbp)     ;比较第二个输入整数和正确口令
40265f: jne    4025ea <secret_phase+0x2a>     ;若不相等,则跳转失败路径
```

```
00000000004025b0 <secret_mix>:
  4025b0: endbr64
  4025b4: mov    %edi,%eax     ;将输入复制到%eax
  4025b6: ror    $0x7,%eax     ;将32位输入循环右移7位
  4025b9: xor    $0xf13d,%eax     ;将结果与0xf13d异或
  4025be: ret    ;返回计算结果
  4025bf: nop
```

```
402661: shr    $0x7,%esi     ;%esi=A>>7
402664: lea    0xc275(%r8,%rcx,1),%edx     ;%edx=B+v0+0xc275,保留低32位
40266c: add    %esi,%edx     ;再加上A>>7
40266e: xor    %edx,%eax     ;将结果与v1异或,得到v2的正确值
402670: cmp    -0x24(%rbp),%eax     ;比较第三个整数与正确口令
402673: sete   %dl     ;相等则%dl=1,否则设为0
402676: jmp    4025ea <secret_phase+0x2a>     ;进入公共返回路径,返回验证结果
40267b: nopl   0x0(%rax,%rax,1)
```

Seed 42 验证

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

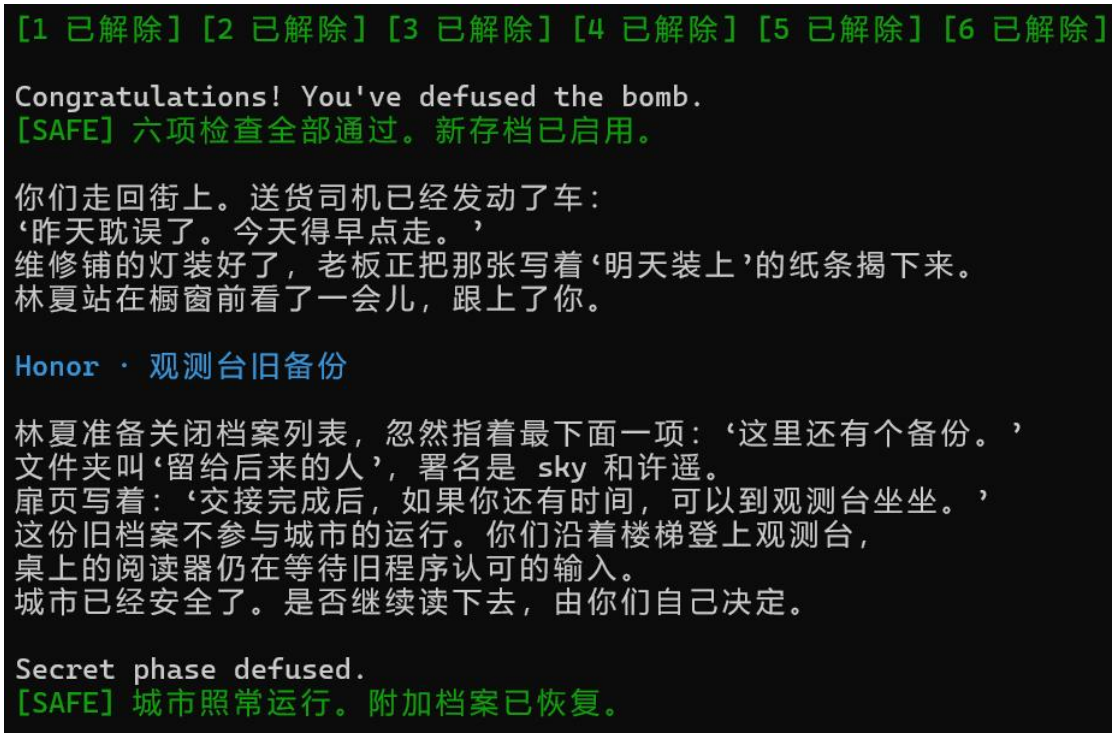
Honor · 观测台旧备份

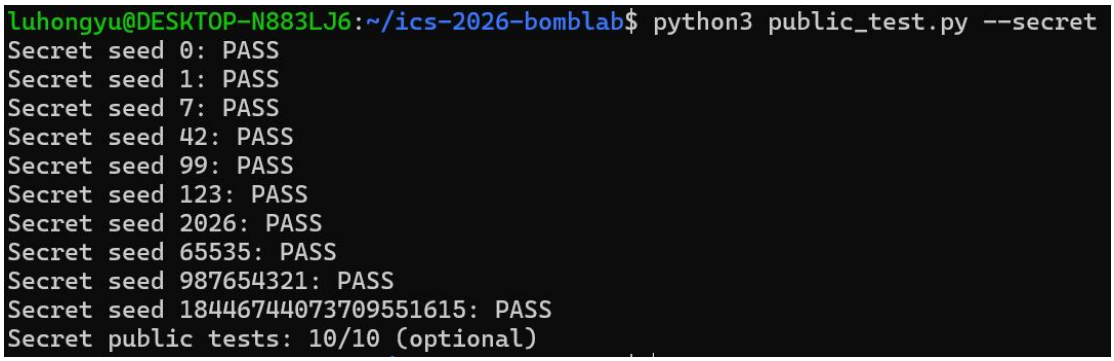
林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

```
luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10 (optional)
```

Seed 43 验证


[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]
Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。
你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。
Honor · 观测台旧备份
林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。
Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。


luhongyu@DESKTOP-N883LJ6:~/ics-2026-bomblab$ python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10 (optional)